

ASSESSMENT

Section A — Concept Application

Q.1 Which development methodology would you recommend for this project, and why? Identify two specific risks the team would face if they chose the wrong methodology for this context.

Ans.

For this digital wallet project, I would recommend Agile methodology because the requirements are not fixed and may change frequently. Agile works in short development cycles called sprints, so the team can build and deliver useful features every two weeks. The client can test these features, give feedback, and the team can make improvements in the next sprint.

If the team chooses the wrong methodology, such as Waterfall, two major risks are:

1. Requirement mismatch: Changes in the client's needs may become difficult and expensive to implement after development has progressed.
2. Delayed problem detection: Since working features are reviewed late in Waterfall, errors or missing requirements may be discovered only near the end of the project.

Therefore, Agile fits this project because it provides flexibility, continuous feedback, and regular delivery of working features.

Q.5 Explain why an array is the correct data structure for this task compared to using 30 separate variables. Describe the two-step logic your program would follow: first to compute the average, then to identify the above-average days — and explain why both steps cannot be done in a single loop.

Ans.

1. Why is an Array the Correct Data Structure?

An array is a collection of elements of the same data type stored under a single name. In this program, we need to store rainfall readings for 30 days, so we can use an array of 30 elements.

For example: `float rainfall[30];`

- It stores all 30 days' rainfall readings under one variable name.
- It reduces the need to create 30 separate variables.
- We can easily access each day's rainfall using an index.
- We can use loops to calculate the average and compare rainfall readings efficiently.

2. Two-Step Logic of the Program

Step 1: Calculate the Monthly Average

- Store the rainfall readings of all 30 days in the array.
- Add all 30 readings using a loop.
- Divide the total rainfall by 30 to calculate the average.

Formula:

$$\text{Average} = \frac{\text{Total rainfall}}{30}$$

Step 2: Identify Above-Average Days

- Use another loop to check each day's rainfall.
- Compare each reading with the calculated average.
- If `rainfall[i] > average`, display that day's number and rainfall reading.

3. Why Are Two Loops Required?

Two loops are required because the average must be calculated before identifying the days with above-average rainfall.

In the first loop, we calculate the total rainfall and then find the average. In the second loop, we compare each day's rainfall with that average.

Both steps cannot normally be completed in a single pass because the final average is not known until all 30 readings have been processed. Therefore, we need the average first to make the correct comparisons.

Q.6 Explain why the program crashes on an empty string input and describe the check your pointer-based solution must perform before dereferencing the pointer. How does traversing a string using a pointer differ from traversing it using array index notation, and which approach makes the empty-input bug easier to catch?

Ans.

1. Why does the program crash on empty string input?

When the user presses Enter without typing anything, the string is empty. In C, an empty string contains only the null character '\0'.

If the program tries to access a character through an invalid or uninitialized pointer, it may crash. Therefore, the pointer must be initialized to point to a valid string before accessing its characters.

2. What check should be performed before dereferencing the pointer?

Before dereferencing the pointer, we should check that the pointer is not NULL and that the character it points to is not the null character '\0'.

Example:

```
if (ptr != NULL && *ptr != '\0')
{
    printf("%c", *ptr);
}
```

Here, `ptr != NULL` checks that the pointer is valid, and `*ptr != '\0'` checks that the string is not empty at the current position.

3. Difference between pointer traversal and array index notation

Pointer traversal: A pointer accesses characters using dereferencing, such as `*ptr`, and moves to the next character using `ptr++`.

Array index notation: Characters are accessed using an index, such as `str[i]`, and the index increases using `i++`.

Both methods can traverse a string, but pointer traversal makes it important to check pointer validity and the null terminator before accessing characters. Array indexing can also cause errors if the index is not checked.